based on the paravirt_ops interface, the patches were only progressively merged into the official kernel. The merge was completed in Linux 3.0.

➨ https://wiki.xenproject.org/wiki/XenParavirtOps

Since *Jessie* is based on version 3.16 of the Linux kernel, the standard *linux-image-686-pae* and *linux-image-amd64* packages include the necessary code, and the distribution-specific patching that was required for *Squeeze* and earlier versions of Debian is no more.

➨ https://wiki.xenproject.org/wiki/Xen_Kernel_Feature_Matrix

CULTURE

Xen and non-Linux kernels

Xen requires modifications to all the operating systems one wants to run on it; not all kernels have the same level of maturity in this regard. Many are fully-functional, both as dom0 and domU: Linux 3.0 and later, NetBSD 4.0 and later, and OpenSolaris. Others only work as a domU. You can check the status of each operating system in the Xen wiki:

➨ https://wiki.xenproject.org/wiki/Dom0_Kernels_for_Xen

➨ https://wiki.xenproject.org/wiki/DomU_Support_for_Xen

However, if Xen can rely on the hardware functions dedicated to virtualization (which are only present in more recent processors), even non-modified operating systems can run as domU (including Windows).

NOTE

Architectures compatible with Xen

Xen is currently only available for the i386, amd64, arm64 and armhf architectures.

Using Xen under Debian requires three components:

- The hypervisor itself. According to the available hardware, the appropriate package will be either *xen-hypervisor-4.11-amd64*, *xen-hypervisor-4.11-armhf*, or *xen-hypervisor-4.11-arm64*.
- A kernel that runs on that hypervisor. Any kernel more recent than 3.0 will do, including the 4.19 version present in *Buster*.
- The i386 architecture also requires a standard library with the appropriate patches taking advantage of Xen; this is in the *libc6-xen* package.

The hypervisor also brings *xen-utils-4.11*, which contains tools to control the hypervisor from the dom0. This in turn brings the appropriate standard library. During the installation of all that, configuration scripts also create a new entry in the GRUB bootloader menu, so as to start the chosen kernel in a Xen dom0. Note, however, that this entry is not usually set to be the first one in the list, but it will be selected by default.

Once these prerequisites are installed, the next step is to test the behavior of the dom0 by itself; this involves a reboot to the hypervisor and the Xen kernel. The system should boot in its standard fashion, with a few extra messages on the console during the early initialization steps.

Now is the time to actually install useful systems on the domU systems, using the tools from *xen-tools*. This package provides the `xen-create-image` command, which largely automates the task. The only mandatory parameter is `--hostname`, giving a name to the domU; other options

are important, but they can be stored in the `/etc/xen-tools/xen-tools.conf` configuration file, and their absence from the command line doesn't trigger an error. It is therefore important to either check the contents of this file before creating images, or to use extra parameters in the `xen-create-image` invocation. Important parameters of note include the following:

- --memory, to specify the amount of RAM dedicated to the newly created system;
- --size and --swap, to define the size of the "virtual disks" available to the domU;
- --debootstrap-cmd, to specify the which debootstrap command is used. The default is `debootstrap` if debootstrap and cdebootstrap are installed. In that case, the --dist option will also most often be used (with a distribution name such as *buster*).

GOING FURTHER
Installing a non-Debian system in a domU

In case of a non-Linux system, care should be taken to define the kernel the domU must use, using the `--kernel` option.

- --dhcp states that the domU's network configuration should be obtained by DHCP while --ip allows defining a static IP address.
- Lastly, a storage method must be chosen for the images to be created (those that will be seen as hard disk drives from the domU). The simplest method, corresponding to the --dir option, is to create one file on the dom0 for each device the domU should be provided. For systems using LVM, the alternative is to use the --lvm option, followed by the name of a volume group; `xen-create-image` will then create a new logical volume inside that group, and this logical volume will be made available to the domU as a hard disk drive.

NOTE
Storage in the domU

Entire hard disks can also be exported to the domU, as well as partitions, RAID arrays or pre-existing LVM logical volumes. These operations are not automated by `xen-create-image`, however, so editing the Xen image's configuration file is in order after its initial creation with `xen-create-image`.

Once these choices are made, we can create the image for our future Xen domU:

```
# xen-create-image --hostname testxen --dhcp --dir /srv/testxen --size=2G --dist=
    ➥ buster --role=udev

[...]
eneral Information
--------------------
Hostname       :  testxen
Distribution   :  buster
Mirror         :  http://deb.debian.org/debian
Partitions     :  swap            512M  (swap)
                  /               2G    (ext4)
Image type     :  sparse
Memory size    :  256M
Kernel path    :  /boot/vmlinuz-4.19.0-5-amd64
Initrd path    :  /boot/initrd.img-4.19.0-5-amd64
[...]
Logfile produced at:
         /var/log/xen-tools/testxen.log
```

```
Installation Summary
---------------------
Hostname        :  testxen
Distribution    :  buster
MAC Address     :  00:16:3E:0C:74:2F
IP Address(es)  :  dynamic
SSH Fingerprint :  SHA256:PuAGX4/4S07Xzh1u0Cl2tL04EL5udf9ajvvbufBrfvU (DSA)
SSH Fingerprint :  SHA256:ajFTX54eakzolyzmZku/ihq/BK6KYsz5MewJ98BM5co (ECDSA)
SSH Fingerprint :  SHA256:/sFov86b+rD/bRSJoHKbiMqzGFiwgZulEwpzsiw6aSc (ED25519)
SSH Fingerprint :  SHA256:/NJg/CcoVj+OLE/cL3yyJINStnla7YkHKe3/xEdVGqc (RSA)
Root Password   :  EwmQMHtywY9zsRBpqQuxZTb
```

We now have a virtual machine, but it is currently not running (and therefore only using space on the dom0's hard disk). Of course, we can create more images, possibly with different parameters.

Before turning these virtual machines on, we need to define how they'll be accessed. They can of course be considered as isolated machines, only accessed through their system console, but this rarely matches the usage pattern. Most of the time, a domU will be considered as a remote server, and accessed only through a network. However, it would be quite inconvenient to add a network card for each domU; which is why Xen allows creating virtual interfaces, that each domain can see and use in a standard way. Note that these cards, even though they're virtual, will only be useful once connected to a network, even a virtual one. Xen has several network models for that:

- The simplest model is the *bridge* model; all the eth0 network cards (both in the dom0 and the domU systems) behave as if they were directly plugged into an Ethernet switch.
- Then comes the *routing* model, where the dom0 behaves as a router that stands between the domU systems and the (physical) external network.
- Finally, in the *NAT* model, the dom0 is again between the domU systems and the rest of the network, but the domU systems are not directly accessible from outside, and traffic goes through some network address translation on the dom0.

These three networking nodes involve a number of interfaces with unusual names, such as `vif*`, `veth*`, `peth*` and `xenbr0`. The Xen hypervisor arranges them in whichever layout has been defined, under the control of the user-space tools. Since the NAT and routing models are only adapted to particular cases, we will only address the bridging model.

The standard configuration of the Xen packages does not change the system-wide network configuration. However, the `xend` daemon is configured to integrate virtual network interfaces into any pre-existing network bridge (with `xenbr0` taking precedence if several such bridges exist). We must therefore set up a bridge in `/etc/network/interfaces` (which requires installing the *bridge-utils* package, which is why the *xen-utils-4.11* package recommends it) to replace the existing eth0 entry:

```
auto xenbr0
iface xenbr0 inet dhcp
```

```
    bridge_ports eth0
    bridge_maxwait 0
```

After rebooting to make sure the bridge is automatically created, we can now start the domU with the Xen control tools, in particular the `xl` command. This command allows different manipulations on the domains, including listing them and, starting/stopping them. You might need to increase the default memory by editing the variable memory from configuration file (in this case, `/etc/xen/testxen.cfg`). Here we have set it to 1024 (megabytes).

```
# xl list
Name                                        ID   Mem VCPUs        State        Time(s
    ➥ )
Domain-0                                     0  1894     2     r-----      63.5
# xl create /etc/xen/testxen.cfg
Parsing config from /etc/xen/testxen.cfg
# xl list
Name                                        ID   Mem VCPUs        State        Time(s
    ➥ )
Domain-0                                     0  1505     2     r-----     100.0
testxen                                     13  1024     0     --p---       0.0
```

TOOL

Choice of toolstacks to manage Xen VM

In Debian 7 and older releases, `xm` was the reference command line tool to use to manage Xen virtual machines. It has now been replaced by `xl` which is mostly backwards compatible. But those are not the only available tools: `virsh` of libvirt and `xe` of XenServer's XAPI (commercial offering of Xen) are alternative tools.

CAUTION

Only one domU per image!

While it is of course possible to have several domU systems running in parallel, they will all need to use their own image, since each domU is made to believe it runs on its own hardware (apart from the small slice of the kernel that talks to the hypervisor). In particular, it isn't possible for two domU systems running simultaneously to share storage space. If the domU systems are not run at the same time, it is, however, quite possible to reuse a single swap partition, or the partition hosting the `/home` filesystem.

Note that the `testxen` domU uses real memory taken from the RAM that would otherwise be available to the dom0, not simulated memory. Care should therefore be taken, when building a server meant to host Xen instances, to provision the physical RAM accordingly.

Voilà! Our virtual machine is starting up. We can access it in one of two modes. The usual way is to connect to it "remotely" through the network, as we would connect to a real machine; this will usually require setting up either a DHCP server or some DNS configuration. The other way, which may be the only way if the network configuration was incorrect, is to use the `hvc0` console, with the `xl console` command:

```
# xl console testxen
[...]
```

```
Debian GNU/Linux 10 testxen hvc0

testxen login:
```

One can then open a session, just like one would do if sitting at the virtual machine's keyboard. Detaching from this console is achieved through the Control+] key combination.

TIP

Getting the console straight away

Sometimes one wishes to start a domU system and get to its console straight away; this is why the `xl create` command takes a `-c` switch. Starting a domU with this switch will display all the messages as the system boots.

TOOL

OpenXenManager

OpenXenManager (in the *openxenmanager* package) is a graphical interface allowing remote management of Xen domains via Xen's API. It can thus control Xen domains remotely. It provides most of the features of the `xl` command.

Once the domU is up, it can be used just like any other server (since it is a GNU/Linux system after all). However, its virtual machine status allows some extra features. For instance, a domU can be temporarily paused then resumed, with the `xl pause` and `xl unpause` commands. Note that even though a paused domU does not use any processor power, its allocated memory is still in use. It may be interesting to consider the `xl save` and `xl restore` commands: saving a domU frees the resources that were previously used by this domU, including RAM. When restored (or unpaused, for that matter), a domU doesn't even notice anything beyond the passage of time. If a domU was running when the dom0 is shut down, the packaged scripts automatically save the domU, and restore it on the next boot. This will of course involve the standard inconvenience incurred when hibernating a laptop computer, for instance; in particular, if the domU is suspended for too long, network connections may expire. Note also that Xen is so far incompatible with a large part of ACPI power management, which precludes suspending the host (dom0) system.

DOCUMENTATION

`xl` options

Most of the `xl` subcommands expect one or more arguments, often a domU name. These arguments are well described in the `xl(1)` manual page.

Halting or rebooting a domU can be done either from within the domU (with the `shutdown` command) or from the dom0, with `xl shutdown` or `xl reboot`.

GOING FURTHER

Advanced Xen

Xen has many more features than we can describe in these few paragraphs. In particular, the system is very dynamic, and many parameters for one domain (such as the amount of allocated memory, the visible hard drives, the behavior of the task scheduler, and so on) can be adjusted even when that domain is running. A domU can even be migrated across servers without being shut down, and without losing its network connections! For all these advanced aspects, the primary source of information is the official Xen documentation.

➨ https://xenproject.org/help/documentation/

12.2.2. LXC

Even though it is used to build "virtual machines", LXC is not, strictly speaking, a virtualization system, but a system to isolate groups of processes from each other even though they all run on the same host. It takes advantage of a set of recent evolutions in the Linux kernel, collectively known as *control groups*, by which different sets of processes called "groups" have different views of certain aspects of the overall system. Most notable among these aspects are the process identifiers, the network configuration, and the mount points. Such a group of isolated processes will not have any access to the other processes in the system, and its accesses to the filesystem can be restricted to a specific subset. It can also have its own network interface and routing table, and it may be configured to only see a subset of the available devices present on the system.

These features can be combined to isolate a whole process family starting from the `init` process, and the resulting set looks very much like a virtual machine. The official name for such a setup is a "container" (hence the LXC moniker: *LinuX Containers*), but a rather important difference with "real" virtual machines such as provided by Xen or KVM is that there is no second kernel; the container uses the very same kernel as the host system. This has both pros and cons: advantages include excellent performance due to the total lack of overhead, and the fact that the kernel has a global vision of all the processes running on the system, so the scheduling can be more efficient than it would be if two independent kernels were to schedule different task sets. Chief among the inconveniences is the impossibility to run a different kernel in a container (whether a different Linux version or a different operating system altogether).

NOTE

LXC isolation limits

LXC containers do not provide the level of isolation achieved by heavier emulators or virtualizers. In particular:

- since the kernel is shared among the host system and the containers, processes constrained to containers can still access the kernel messages, which can lead to information leaks if messages are emitted by a container;
- for similar reasons, if a container is compromised and a kernel vulnerability is exploited, the other containers may be affected too;
- on the filesystem, the kernel checks permissions according to the numerical identifiers for users and groups; these identifiers may designate different users and groups depending on the container, which should be kept in mind if writable parts of the filesystem are shared among containers.

Since we are dealing with isolation and not plain virtualization, setting up LXC containers is more complex than just running debian-installer on a virtual machine. We will describe a few prerequisites, then go on to the network configuration; we will then be able to actually create the system to be run in the container.

Preliminary Steps

The *lxc* package contains the tools required to run LXC, and must therefore be installed.

LXC also requires the *control groups* configuration system, which is a virtual filesystem to be mounted on `/sys/fs/cgroup`. Since Debian 8 switched to systemd, which also relies on control groups, this is now done automatically at boot time without further configuration.

Network Configuration

The goal of installing LXC is to set up virtual machines; while we could, of course, keep them isolated from the network, and only communicate with them via the filesystem, most use cases involve giving at least minimal network access to the containers. In the typical case, each container will get a virtual network interface, connected to the real network through a bridge. This virtual interface can be plugged either directly onto the host's physical network interface (in which case the container is directly on the network), or onto another virtual interface defined on the host (and the host can then filter or route traffic). In both cases, the *bridge-utils* package will be required.

The simple case is just a matter of editing `/etc/network/interfaces`, moving the configuration for the physical interface (for instance, `eth0`) to a bridge interface (usually `br0`), and configuring the link between them. For instance, if the network interface configuration file initially contains entries such as the following:

```
auto eth0
iface eth0 inet dhcp
```

They should be disabled and replaced with the following:

```
#auto eth0
#iface eth0 inet dhcp

auto br0
iface br0 inet dhcp
  bridge-ports eth0
```

The effect of this configuration will be similar to what would be obtained if the containers were machines plugged into the same physical network as the host. The "bridge" configuration manages the transit of Ethernet frames between all the bridged interfaces, which includes the physical `eth0` as well as the interfaces defined for the containers.

In cases where this configuration cannot be used (for instance, if no public IP addresses can be assigned to the containers), a virtual *tap* interface will be created and connected to the bridge. The equivalent network topology then becomes that of a host with a second network card plugged into a separate switch, with the containers also plugged into that switch. The host must then act as a gateway for the containers if they are meant to communicate with the outside world.

In addition to *bridge-utils*, this "rich" configuration requires the *vde2* package; the `/etc/network/interfaces` file then becomes:

```
# Interface eth0 is unchanged
auto eth0
```

```
iface eth0 inet dhcp

# Virtual interface
auto tap0
iface tap0 inet manual
  vde2-switch -t tap0

# Bridge for containers
auto br0
iface br0 inet static
  bridge-ports tap0
  address 10.0.0.1
  netmask 255.255.255.0
```

The network can then be set up either statically in the containers, or dynamically with DHCP server running on the host. Such a DHCP server will need to be configured to answer queries on the br0 interface.

Setting Up the System

Let us now set up the filesystem to be used by the container. Since this "virtual machine" will not run directly on the hardware, some tweaks are required when compared to a standard filesystem, especially as far as the kernel, devices and consoles are concerned. Fortunately, the *lxc* includes scripts that mostly automate this configuration. For instance, the following commands (which require the *debootstrap* and *rsync* packages) will install a Debian container:

```
root@mirwiz:~# lxc-create -n testlxc -t debian
debootstrap is /usr/sbin/debootstrap
Checking cache download in /var/cache/lxc/debian/rootfs-stable-amd64 ...
Downloading debian minimal ...
I: Retrieving Release
I: Retrieving Release.gpg
[...]
Download complete.
Copying rootfs to /var/lib/lxc/testlxc/rootfs...
[...]
root@mirwiz:~#
```

Note that the filesystem is initially created in `/var/cache/lxc`, then moved to its destination directory. This allows creating identical containers much more quickly, since only copying is then required.

Note that the Debian template creation script accepts an `--arch` option to specify the architecture of the system to be installed and a `--release` option if you want to install something else than the current stable release of Debian. You can also set the MIRROR environment variable to point to a local Debian mirror.

The newly-created filesystem now contains a minimal Debian system, and by default the container has no network interface (besides the loopback one). Since this is not really wanted, we will edit the container's configuration file (`/var/lib/lxc/testlxc/config`) and add a few lxc.network.* entries:

```
lxc.net.0.type = veth
lxc.net.0.flags = up
lxc.net.0.link = br0
lxc.net.0.hwaddr = 4a:49:43:49:79:20
```

These entries mean, respectively, that a virtual interface will be created in the container; that it will automatically be brought up when said container is started; that it will automatically be connected to the br0 bridge on the host; and that its MAC address will be as specified. Should this last entry be missing or disabled, a random MAC address will be generated.

Another useful entry in that file is the setting of the hostname:

```
lxc.uts.name = testlxc
```

Starting the Container

Now that our virtual machine image is ready, let's start the container with `lxc-start --daemon --name=testlxc`.

In LXC releases following 2.0.8, root passwords are not set by default. We can set one running `lxc-attach -n testlxc` *`passwd`*. Now we can login:

```
root@mirwiz:~# lxc-console -n testlxc
Debian GNU/Linux 9 testlxc console

testlxc login: root
Password:
Linux testlxc 4.19.0-5-amd64 #1 SMP Debian 4.19.37-5 (2019-06-19) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@testlxc:~# ps auxwf
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.2  56736  6608 ?        Ss   09:28   0:00 /sbin/init
root        32  0.0  0.1  46096  4680 ?        Ss   09:28   0:00 /lib/systemd/systemd-journald
root        75  0.0  0.1  67068  3328 console  Ss   09:28   0:00 /bin/login --
root        82  0.0  0.1  19812  3664 console  S    09:30   0:00  \_ -bash
root        88  0.0  0.1  38308  3176 console  R+   09:31   0:00      \_ ps auxwf
root        76  0.0  0.1  69956  5636 ?        Ss   09:28   0:00 /usr/sbin/sshd -D
root@testlxc:~#
```

We are now in the container; our access to the processes is restricted to only those started from the container itself, and our access to the filesystem is similarly restricted to the dedicated subset of the full filesystem (`/var/lib/lxc/testlxc/rootfs`). We can exit the console with Control+a q.

Note that we ran the container as a background process, thanks to the `--daemon` option of `lxc-start`. We can interrupt the container with a command such as `lxc-stop --name=testlxc`.

The *lxc* package contains an initialization script that can automatically start one or several containers when the host boots (it relies on `lxc-autostart` which starts containers whose lxc.start.auto option is set to 1). Finer-grained control of the startup order is possible with lxc.start.order and lxc.group: by default, the initialization script first starts containers which are part of the onboot group and then the containers which are not part of any group. In both cases, the order within a group is defined by the lxc.start.order option.

GOING FURTHER

Mass virtualization

Since LXC is a very lightweight isolation system, it can be particularly adapted to massive hosting of virtual servers. The network configuration will probably be a bit more advanced than what we described above, but the "rich" configuration using `tap` and `veth` interfaces should be enough in many cases.

It may also make sense to share part of the filesystem, such as the `/usr` and `/lib` subtrees, so as to avoid duplicating the software that may need to be common to several containers. This will usually be achieved with `lxc.mount.entry` entries in the containers configuration file. An interesting side-effect is that the processes will then use less physical memory, since the kernel is able to detect that the programs are shared. The marginal cost of one extra container can then be reduced to the disk space dedicated to its specific data, and a few extra processes that the kernel must schedule and manage.

We haven't described all the available options, of course; more comprehensive information can be obtained from the `lxc(7)` and `lxc.container.conf(5)` manual pages and the ones they reference.

12.2.3. Virtualization with KVM

KVM, which stands for *Kernel-based Virtual Machine*, is first and foremost a kernel module providing most of the infrastructure that can be used by a virtualizer, but it is not a virtualizer by itself. Actual control for the virtualization is handled by a QEMU-based application. Don't worry if this section mentions `qemu-*` commands: it is still about KVM.

Unlike other virtualization systems, KVM was merged into the Linux kernel right from the start. Its developers chose to take advantage of the processor instruction sets dedicated to virtualization (Intel-VT and AMD-V), which keeps KVM lightweight, elegant and not resource-hungry. The counterpart, of course, is that KVM doesn't work on any computer but only on those with appropriate processors. For x86-based computers, you can verify that you have such a processor by looking for "vmx" or "svm" in the CPU flags listed in `/proc/cpuinfo`.

With Red Hat actively supporting its development, KVM has more or less become the reference for Linux virtualization.

Preliminary Steps

Unlike such tools as VirtualBox, KVM itself doesn't include any user-interface for creating and managing virtual machines. The *qemu-kvm* package only provides an executable able to start a virtual machine, as well as an initialization script that loads the appropriate kernel modules.

Fortunately, Red Hat also provides another set of tools to address that problem, by developing the *libvirt* library and the associated *virtual machine manager* tools. libvirt allows managing virtual machines in a uniform way, independently of the virtualization system involved behind the scenes (it currently supports QEMU, KVM, Xen, LXC, OpenVZ, VirtualBox, VMWare and UML). `virtual-manager` is a graphical interface that uses libvirt to create and manage virtual machines.

We first install the required packages, with `apt-get install libvirt-clients libvirt-daemon-system qemu-kvm virtinst virt-manager virt-viewer`. *libvirt-daemon-system* provides the `libvirtd` daemon, which allows (potentially remote) management of the virtual machines running of the host, and starts the required VMs when the host boots. *libvirt-clients* provides the `virsh` command-line tool, which allows controlling the `libvirtd`-managed machines.

The *virtinst* package provides `virt-install`, which allows creating virtual machines from the command line. Finally, *virt-viewer* allows accessing a VM's graphical console.

Network Configuration

Just as in Xen and LXC, the most frequent network configuration involves a bridge grouping the network interfaces of the virtual machines (see section 12.2.2.2, "Network Configuration" page 357).

Alternatively, and in the default configuration provided by KVM, the virtual machine is assigned a private address (in the 192.168.122.0/24 range), and NAT is set up so that the VM can access the outside network.

The rest of this section assumes that the host has an `eth0` physical interface and a `br0` bridge, and that the former is connected to the latter.

Installation with `virt-install`

Creating a virtual machine is very similar to installing a normal system, except that the virtual machine's characteristics are described in a seemingly endless command line.

Practically speaking, this means we will use the Debian installer, by booting the virtual machine on a virtual DVD-ROM drive that maps to a Debian DVD image stored on the host system. The VM will export its graphical console over the VNC protocol (see section 9.2.2, "Using Remote Graphical Desktops" page 212 for details), which will allow us to control the installation process.

We first need to tell libvirtd where to store the disk images, unless the default location (`/var/lib/libvirt/images/`) is fine.

```
root@mirwiz:~# mkdir /srv/kvm
root@mirwiz:~# virsh pool-create-as srv-kvm dir --target /srv/kvm
Pool srv-kvm created

root@mirwiz:~#
```

TIP

Add your user to the libvirt group

All samples in this section assume that you are running commands as root. Effectively, if you want to control a local libvirt daemon, you need either to be root or to be a member of the `libvirt` group (which is not the case by default). Thus if you want to avoid using root rights too often, you can add yourself to the `libvirt` group and run the various commands under your user identity.

Let us now start the installation process for the virtual machine, and have a closer look at `virt-install`'s most important options. This command registers the virtual machine and its parameters in libvirtd, then starts it so that its installation can proceed.

```
# virt-install --connect qemu:///system ❶
               --virt-type kvm ❷
               --name testkvm ❸
               --memory 1024 ❹
               --disk /srv/kvm/testkvm.qcow,format=qcow2,size=10 ❺
               --cdrom /srv/isos/debian-10.2.0-amd64-netinst.iso ❻
               --network bridge=virbr0 ❼
               --graphics vnc ❽
               --os-type linux ❾
               --os-variant debian10

Starting install...
Allocating 'testkvm.qcow'              |  10 GB     00:00
```

❶ The --connect option specifies the "hypervisor" to use. Its form is that of an URL containing a virtualization system (xen://, qemu://, lxc://, openvz://, vbox://, and so on) and the machine that should host the VM (this can be left empty in the case of the local host). In addition to that, and in the QEMU/KVM case, each user can manage virtual machines working with restricted permissions, and the URL path allows differentiating "system" machines (/system) from others (/session).

❷ Since KVM is managed the same way as QEMU, the --virt-type kvm allows specifying the use of KVM even though the URL looks like QEMU.

❸ The --name option defines a (unique) name for the virtual machine.

❹ The --memory option allows specifying the amount of RAM (in MB) to allocate for the virtual machine.

5. The --disk specifies the location of the image file that is to represent our virtual machine's hard disk; that file is created, unless present, with a size (in GB) specified by the size parameter. The format parameter allows choosing among several ways of storing the image file. The default format (qcow2) allows starting with a small file that only grows when the virtual machine starts actually using space.

6. The --cdrom option is used to indicate where to find the optical disk to use for installation. The path can be either a local path for an ISO file, an URL where the file can be obtained, or the device file of a physical CD-ROM drive (i.e. /dev/cdrom).

7. The --network specifies how the virtual network card integrates in the host's network configuration. The default behavior (which we explicitly forced in our example) is to integrate it into any pre-existing network bridge. If no such bridge exists, the virtual machine will only reach the physical network through NAT, so it gets an address in a private subnet range (192.168.122.0/24).

8. --graphics vnc states that the graphical console should be made available using VNC. The default behavior for the associated VNC server is to only listen on the local interface; if the VNC client is to be run on a different host, establishing the connection will require setting up an SSH tunnel (see section 9.2.1.3, "Creating Encrypted Tunnels with Port Forwarding" page 210). Alternatively, --graphics vnc,listen=0.0.0.0 can be used so that the VNC server is accessible from all interfaces; note that if you do that, you really should design your firewall accordingly.

9. The --os-type and --os-variant options allow optimizing a few parameters of the virtual machine, based on some of the known features of the operating system mentioned there.

At this point, the virtual machine is running, and we need to connect to the graphical console to proceed with the installation process. If the previous operation was run from a graphical desktop environment, this connection should be automatically started. If not, or if we operate remotely, `virt-viewer` can be run from any graphical environment to open the graphical console (note that the root password of the remote host is asked twice because the operation requires 2 SSH connections):

```
$ virt-viewer --connect qemu+ssh://root@server/system testkvm
root@server's password:
root@server's password:
```

When the installation process ends, the virtual machine is restarted, now ready for use.

Managing Machines with `virsh`

Now that the installation is done, let us see how to handle the available virtual machines. The first thing to try is to ask `libvirtd` for the list of the virtual machines it manages:

```
# virsh -c qemu:///system list --all
 Id Name                 State
----------------------------------
  8 testkvm              shut off
```

Let's start our test virtual machine:

```
# virsh -c qemu:///system start testkvm
Domain testkvm started
```

We can now get the connection instructions for the graphical console (the returned VNC display can be given as parameter to `vncviewer`):

```
# virsh -c qemu:///system vncdisplay testkvm
127.0.0.1:0
```

Other available `virsh` subcommands include:

- reboot to restart a virtual machine;
- shutdown to trigger a clean shutdown;
- destroy, to stop it brutally;
- suspend to pause it;
- resume to unpause it;
- autostart to enable (or disable, with the --disable option) starting the virtual machine automatically when the host starts;
- undefine to remove all traces of the virtual machine from `libvirtd`.

All these subcommands take a virtual machine identifier as a parameter.

Installing an RPM based system in Debian with yum

If the virtual machine is meant to run a Debian (or one of its derivatives), the system can be initialized with `debootstrap`, as described above. But if the virtual machine is to be installed with an RPM-based system (such as Fedora, CentOS or Scientific Linux), the setup will need to be done using the `yum` utility (available in the package of the same name).

The procedure requires using `rpm` to extract an initial set of files, including notably `yum` configuration files, and then calling `yum` to extract the remaining set of packages. But since we call `yum` from outside the chroot, we need to make some temporary changes. In the sample below, the target chroot is `/srv/centos`.

```
# rootdir="/srv/centos"
# mkdir -p "$rootdir" /etc/rpm
# echo "%_dbpath /var/lib/rpm" > /etc/rpm/macros.dbpath
```

```
# wget http://mirror.centos.org/centos/7/os/x86_64/Packages/centos-release
    ➥ -7-6.1810.2.el7.centos.x86_64.rpm
# rpm --nodeps --root "$rootdir" -i centos-release-7-6.1810.2.el7.centos.x86_64.rpm
rpm: RPM should not be used directly install RPM packages, use Alien instead!
rpm: However assuming you know what you are doing...
warning: centos-release-7-6.1810.2.el7.centos.x86_64.rpm: Header V3 RSA/SHA256
    ➥ Signature, key ID f4a80eb5: NOKEY
# sed -i -e "s,gpgkey=file:///etc/,gpgkey=file://${rootdir}/etc/,g" $rootdir/etc/yum.
    ➥ repos.d/*.repo
# yum --assumeyes --installroot $rootdir groupinstall core
[...]
# sed -i -e "s,gpgkey=file://${rootdir}/etc/,gpgkey=file:///etc/,g" $rootdir/etc/yum.
    ➥ repos.d/*.repo
```

12.3. Automated Installation

The Falcot Corp administrators, like many administrators of large IT services, need tools to install (or reinstall) quickly, and automatically if possible, their new machines.

These requirements can be met by a wide range of solutions. On the one hand, generic tools such as SystemImager handle this by creating an image based on a template machine, then deploy that image to the target systems; at the other end of the spectrum, the standard Debian installer can be preseeded with a configuration file giving the answers to the questions asked during the installation process. As a sort of middle ground, a hybrid tool such as FAI (*Fully Automatic Installer*) installs machines using the packaging system, but it also uses its own infrastructure for tasks that are more specific to massive deployments (such as starting, partitioning, configuration and so on).

Each of these solutions has its pros and cons: SystemImager works independently from any particular packaging system, which allows it to manage large sets of machines using several distinct Linux distributions. It also includes an update system that doesn't require a reinstallation, but this update system can only be reliable if the machines are not modified independently; in other words, the user must not update any software on their own, or install any other software. Similarly, security updates must not be automated, because they have to go through the centralized reference image maintained by SystemImager. This solution also requires the target machines to be homogeneous, otherwise many different images would have to be kept and managed (an i386 image won't fit on a powerpc machine, and so on).

On the other hand, an automated installation using debian-installer can adapt to the specifics of each machine: the installer will fetch the appropriate kernel and software packages from the relevant repositories, detect available hardware, partition the whole hard disk to take advantage of all the available space, install the corresponding Debian system, and set up an appropriate bootloader. However, the standard installer will only install standard Debian versions, with the base system and a set of pre-selected "tasks"; this precludes installing a particular system with non-packaged applications. Fulfilling this particular need requires customizing the installer...

Fortunately, the installer is very modular, and there are tools to automate most of the work required for this customization, most importantly simple-CDD (CDD being an acronym for *Custom Debian Derivative*). Even the simple-CDD solution, however, only handles initial installations; this is usually not a problem since the APT tools allow efficient deployment of updates later on.

We will only give a rough overview of FAI, and skip SystemImager altogether (which is no longer in Debian), in order to focus more intently on debian-installer and simple-CDD, which are more interesting in a Debian-only context.

12.3.1. Fully Automatic Installer (FAI)

Fully Automatic Installer is probably the oldest automated deployment system for Debian, which explains its status as a reference; but its very flexible nature only just compensates for the complexity it involves.

FAI requires a server system to store deployment information and allow target machines to boot from the network. This server requires the *fai-server* package (or *fai-quickstart*, which also brings the required elements for a standard configuration).

FAI uses a specific approach for defining the various installable profiles. Instead of simply duplicating a reference installation, FAI is a full-fledged installer, fully configurable via a set of files and scripts stored on the server; the default location `/srv/fai/config/` is not automatically created, so the administrator needs to create it along with the relevant files. Most of the times, these files will be customized from the example files available in the documentation for the *fai-doc* package, more particularly the `/usr/share/doc/fai-doc/examples/simple/` directory.

Once the profiles are defined, the `fai-setup` command generates the elements required to start a FAI installation; this mostly means preparing or updating a minimal system (NFS-root) used during installation. An alternative is to generate a dedicated boot CD with `fai-cd`.

Creating all these configuration files requires some understanding of the way FAI works. A typical installation process is made of the following steps:

- fetching a kernel from the network, and booting it;
- mounting the root filesystem from NFS;
- executing `/usr/sbin/fai`, which controls the rest of the process (the next steps are therefore initiated by this script);
- copying the configuration space from the server into `/fai/`;
- running `fai-class`. The `/fai/class/[0-9][0-9]*` scripts are executed in turn, and return names of "classes" that apply to the machine being installed; this information will serve as a base for the following steps. This allows for some flexibility in defining the services to be installed and configured.
- fetching a number of configuration variables, depending on the relevant classes;
- partitioning the disks and formatting the partitions, based on information provided in `/fai/disk_config/`*`class`*;

- mounting said partitions;
- installing the base system;
- preseeding the Debconf database with `fai-debconf`;
- fetching the list of available packages for APT;
- installing the packages listed in `/fai/package_config/`*`class`*;
- executing the post-configuration scripts, `/fai/scripts/`*`class`*`/[0-9][0-9]*`;
- recording the installation logs, unmounting the partitions, and rebooting.

12.3.2. Preseeding Debian-Installer

At the end of the day, the best tool to install Debian systems should logically be the official Debian installer. This is why, right from its inception, debian-installer has been designed for automated use, taking advantage of the infrastructure provided by *debconf*. The latter allows, on the one hand, to reduce the number of questions asked (hidden questions will use the provided default answer), and on the other hand, to provide the default answers separately, so that installation can be non-interactive. This last feature is known as *preseeding*.

GOING FURTHER

Debconf with a centralized database

Preseeding allows to provide a set of answers to Debconf questions at installation time, but these answers are static and do not evolve as time passes. Since already-installed machines may need upgrading, and new answers may become required, the `/etc/debconf.conf` configuration file can be set up so that Debconf uses external data sources (such as an LDAP directory server, or a remote file accessed via NFS or Samba). Several external data sources can be defined at the same time, and they complement one another. The local database is still used (for read-write access), but the remote databases are usually restricted to reading. The `debconf.conf(5)` manual page describes all the possibilities in detail (you need the *debconf-doc* package).

Using a Preseed File

There are several places where the installer can get a preseeding file:

- in the initrd used to start the machine; in this case, preseeding happens at the very beginning of the installation, and all questions can be avoided. The file just needs to be called `preseed.cfg` and stored in the initrd root.
- on the boot media (CD or USB key); preseeding then happens as soon as the media is mounted, which means right after the questions about language and keyboard layout. The preseed/file boot parameter can be used to indicate the location of the preseeding file (for instance, `/cdrom/preseed.cfg` when the installation is done off a CD-ROM, or `/hd-media/preseed.cfg` in the USB-key case).
- from the network; preseeding then only happens after the network is (automatically) configured; the relevant boot parameter is then preseed/url=http://*server*/preseed.cfg.

At a glance, including the preseeding file in the initrd looks like the most interesting solution; however, it is rarely used in practice, because generating an installer initrd is rather complex. The other two solutions are much more common, especially since boot parameters provide another way to preseed the answers to the first questions of the installation process. The usual way to save the bother of typing these boot parameters by hand at each installation is to save them into the configuration for `isolinux` (in the CD-ROM case) or `syslinux` (USB key).

Creating a Preseed File

A preseed file is a plain text file, where each line contains the answer to one Debconf question. A line is split across four fields separated by whitespace (spaces or tabs), as in, for instance, d-i mirror/suite string stable:

- the first field is the "owner" of the question; "d-i" is used for questions relevant to the installer, but it can also be a package name for questions coming from Debian packages;
- the second field is an identifier for the question;
- third, the type of question;
- the fourth and last field contains the value for the answer. Note that it must be separated from the third field with a single space; if there are more than one, the following space characters are considered part of the value.

The simplest way to write a preseed file is to install a system by hand. Then `debconf-get-selections --installer` will provide the answers concerning the installer. Answers about other packages can be obtained with `debconf-get-selections`. However, a cleaner solution is to write the preseed file by hand, starting from an example and the reference documentation: with such an approach, only questions where the default answer needs to be overridden can be preseeded; using the priority=critical boot parameter will instruct Debconf to only ask critical questions, and use the default answer for others.

DOCUMENTATION

Installation guide appendix

The installation guide, available online, includes detailed documentation on the use of a preseed file in an appendix. It also includes a detailed and commented sample file, which can serve as a base for local customizations.

➨ `https://www.debian.org/releases/stable/amd64/apb`

➨ `https://www.debian.org/releases/stable/example-preseed.txt`

Creating a Customized Boot Media

Knowing where to store the preseed file is all very well, but the location isn't everything: one must, one way or another, alter the installation boot media to change the boot parameters and add the preseed file.

Booting From the Network When a computer is booted from the network, the server sending the initialization elements also defines the boot parameters. Thus, the change needs to be made in the PXE configuration for the boot server; more specifically, in its `/tftpboot/pxelinux.cfg/default` configuration file. Setting up network boot is a prerequisite; see the Installation Guide for details.

➠ `https://www.debian.org/releases/stable/amd64/ch04s05`

Preparing a Bootable USB Key Once a bootable key has been prepared (see section 4.1.2, "Booting from a USB Key" page 53), a few extra operations are needed. Assuming the key contents are available under `/media/usbdisk/`:

- copy the preseed file to `/media/usbdisk/preseed.cfg`
- edit `/media/usbdisk/syslinux.cfg` and add required boot parameters (see example below).

Example 12.2 *syslinux.cfg file and preseeding parameters*

```
default vmlinuz
append preseed/file=/hd-media/preseed.cfg locale=en_US.UTF-8 keymap=us language=us
    ➥ country=US vga=788 initrd=initrd.gz  --
```

Creating a CD-ROM Image A USB key is a read-write media, so it was easy for us to add a file there and change a few parameters. In the CD-ROM case, the operation is more complex, since we need to regenerate a full ISO image. This task is handled by *debian-cd*, but this tool is rather awkward to use: it needs a local mirror, and it requires an understanding of all the options provided by `/usr/share/debian-cd/CONF.sh`; even then, `make` must be invoked several times. `/usr/share/debian-cd/README` is therefore a very recommended read.

Having said that, debian-cd always operates in a similar way: an "image" directory with the exact contents of the CD-ROM is generated, then converted to an ISO file with a tool such as `genisoimage`, `mkisofs` or `xorriso`. The image directory is finalized after debian-cd's `make image-trees` step. At that point, we insert the preseed file into the appropriate directory (usually `$TDIR/$CODENAME/CD1/`, $TDIR and $CODENAME being parameters defined by the `CONF.sh` configuration file). The CD-ROM uses `isolinux` as its bootloader, and its configuration file must be adapted from what debian-cd generated, in order to insert the required boot parameters (the specific file is `$TDIR/$CODENAME/boot1/isolinux/isolinux.cfg`). Then the "normal" process can be resumed, and we can go on to generating the ISO image with `make image CD=1` (or `make images` if several CD-ROMs are generated).

12.3.3. Simple-CDD: The All-In-One Solution

Simply using a preseed file is not enough to fulfill all the requirements that may appear for large deployments. Even though it is possible to execute a few scripts at the end of the normal installation process, the selection of the set of packages to install is still not quite flexible (basically, only "tasks" can be selected); more important, this only allows installing official Debian packages, and precludes locally-generated ones.

On the other hand, debian-cd is able to integrate external packages, and debian-installer can be extended by inserting new steps in the installation process. By combining these capabilities, it should be possible to create a customized installer that fulfills our needs; it should even be able to configure some services after unpacking the required packages. Fortunately, this is not a mere hypothesis, since this is exactly what Simple-CDD (in the *simple-cdd* package) does.

The purpose of Simple-CDD is to allow anyone to easily create a distribution derived from Debian, by selecting a subset of the available packages, preconfiguring them with Debconf, adding specific software, and executing custom scripts at the end of the installation process. This matches the "universal operating system" philosophy, since anyone can adapt it to their own needs.

Creating Profiles

Simple-CDD defines "profiles" that match the FAI "classes" concept, and a machine can have several profiles (determined at installation time). A profile is defined by a set of `profiles/`*`profile`*`.*` files:

- the `.description` file contains a one-line description for the profile;
- the `.packages` file lists packages that will automatically be installed if the profile is selected;
- the `.downloads` file lists packages that will be stored onto the installation media, but not necessarily installed;
- the `.preseed` file contains preseeding information for Debconf questions (for the installer and/or for packages);
- the `.postinst` file contains a script that will be run at the end of the installation process;
- lastly, the `.conf` file allows changing some Simple-CDD parameters based on the profiles to be included in an image.

The default profile has a particular role, since it is always selected; it contains the bare minimum required for Simple-CDD to work. The only thing that is usually customized in this profile is the simple-cdd/profiles preseed parameter: this allows avoiding the question, introduced by Simple-CDD, about what profiles to install.

Note also that the commands will need to be invoked from the parent directory of the `profiles` directory.

QUICK LOOK

Detailed configuration file

An example of a Simple-CDD configuration file, with all possible parameters, is included in the package (`/usr/share/doc/simple-cdd/examples/simple-cdd.conf.detailed.gz`). This can be used as a starting point when creating a custom configuration file.

Simple-CDD requires many parameters to operate fully. They will most often be gathered in a configuration file, which `build-simple-cdd` can be pointed at with the --conf option, but they can also be specified via dedicated parameters given to `build-simple-cdd`. Here is an overview of how this command behaves, and how its parameters are used:

- the profiles parameter lists the profiles that will be included on the generated CD-ROM image;
- based on the list of required packages, Simple-CDD downloads the appropriate files from the server mentioned in server, and gathers them into a partial mirror (which will later be given to debian-cd);
- the custom packages mentioned in local_packages are also integrated into this local mirror;
- debian-cd is then executed (within a default location that can be configured with the debian_cd_dir variable), with the list of packages to integrate;
- once debian-cd has prepared its directory, Simple-CDD applies some changes to this directory:
  - files containing the profiles are added in a `simple-cdd` subdirectory (that will end up on the CD-ROM);
  - other files listed in the all_extras parameter are also added;
  - the boot parameters are adjusted so as to enable the preseeding. Questions concerning language and country can be avoided if the required information is stored in the language and country variables.
- debian-cd then generates the final ISO image.

Generating an ISO Image

Once we have written a configuration file and defined our profiles, the remaining step is to invoke `build-simple-cdd --conf simple-cdd.conf`. After a few minutes, we get the required image in `images/debian-10-amd64-CD-1.iso`.

12.4. Monitoring

Monitoring is a generic term, and the various involved activities have several goals: on the one hand, following usage of the resources provided by a machine allows anticipating saturation

and the subsequent required upgrades; on the other hand, alerting the administrator as soon as a service is unavailable or not working properly means that the problems that do happen can be fixed sooner.

Munin covers the first area, by displaying graphical charts for historical values of a number of parameters (used RAM, occupied disk space, processor load, network traffic, Apache/MySQL load, and so on). *Nagios* covers the second area, by regularly checking that the services are working and available, and sending alerts through the appropriate channels (e-mails, text messages, and so on). Both have a modular design, which makes it easy to create new plug-ins to monitor specific parameters or services.

ALTERNATIVE

Zabbix, an integrated monitoring tool

Although Munin and Nagios are in very common use, they are not the only players in the monitoring field, and each of them only handles half of the task (graphing on one side, alerting on the other). Zabbix, on the other hand, integrates both parts of monitoring; it also has a web interface for configuring the most common aspects. It has grown by leaps and bounds during the last few years, and can now be considered a viable contender. On the monitoring server, you would install *zabbix-server-pgsql* (or *zabbix-server-mysql*), possibly together with *zabbix-frontend-php* to have a web interface. On the hosts to monitor you would install *zabbix-agent* feeding data back to the server.

➨ https://www.zabbix.com/

ALTERNATIVE

Icinga, a Nagios fork

Spurred by divergences in opinions concerning the development model for Nagios (which is controlled by a company), a number of developers forked Nagios and use Icinga as their new name. Icinga is still compatible — so far — with Nagios configurations and plugins, but it also adds extra features.

➨ https://www.icinga.org/

12.4.1. Setting Up Munin

The purpose of Munin is to monitor many machines; therefore, it quite naturally uses a client/server architecture. The central host — the grapher — collects data from all the monitored hosts, and generates historical graphs.

Configuring Hosts To Monitor

The first step is to install the *munin-node* package. The daemon installed by this package listens on port 4949 and sends back the data collected by all the active plugins. Each plugin is a simple program returning a description of the collected data as well as the latest measured value. Plugins are stored in `/usr/share/munin/plugins/`, but only those with a symbolic link in `/etc/munin/plugins/` are really used.

When the package is installed, a set of active plugins is determined based on the available software and the current configuration of the host. However, this autoconfiguration depends on

a feature that each plugin must provide, and it is usually a good idea to review and tweak the results by hand. Browsing the Plugin Gallery[2] can be interesting even though not all plugins have comprehensive documentation. However, all plugins are scripts and most are rather simple and well-commented. Browsing `/etc/munin/plugins/` is therefore a good way of getting an idea of what each plugin is about and determining which should be removed. Similarly, enabling an interesting plugin found in `/usr/share/munin/plugins/` is a simple matter of setting up a symbolic link with `ln -sf /usr/share/munin/plugins/`*`plugin`*` /etc/munin/plugins/`. Note that when a plugin name ends with an underscore "_", the plugin requires a parameter. This parameter must be stored in the name of the symbolic link; for instance, the "if_" plugin must be enabled with a `if_eth0` symbolic link, and it will monitor network traffic on the eth0 interface.

Once all plugins are correctly set up, the daemon configuration must be updated to describe access control for the collected data. This involves allow directives in the `/etc/munin/munin-node.conf` file. The default configuration is allow `^127\.0\.0\.1$`, and only allows access to the local host. An administrator will usually add a similar line containing the IP address of the grapher host, then restart the daemon with `systemctl restart munin-node`.

GOING FURTHER

Creating local plugins

Munin does include detailed documentation on how plugins should behave, and how to develop new plugins.

➨ http://guide.munin-monitoring.org/en/latest/plugin/writing.html

A plugin is best tested when run in the same conditions as it would be when triggered by munin-node; this can be simulated by running `munin-run `*`plugin`* as root. A potential second parameter given to this command (such as `config`) is passed to the plugin as a parameter.

When a plugin is invoked with the `config` parameter, it must describe itself by returning a set of fields:

```
$ sudo munin-run load config
graph_title Load average
graph_args --base 1000 -l 0
graph_vlabel load
graph_scale no
graph_category system
load.label load
graph_info The load average of the machine describes how
    ➥ many processes are in the run-queue (scheduled to run
    ➥  "immediately").
load.info 5 minute load average
```

The various available fields are described by the "Plugin reference" available as part of the "Munin guide".

➨ https://munin.readthedocs.org/en/latest/reference/plugin.html

[2] ➨ http://gallery.munin-monitoring.org

When invoked without a parameter, the plugin simply returns the last measured values; for instance, executing `sudo munin-run load` could return `load.value 0.12`.

Finally, when a plugin is invoked with the `autoconf` parameter, it should return "yes" (and a 0 exit status) or "no" (with a 1 exit status) according to whether the plugin should be enabled on this host.

Configuring the Grapher

The "grapher" is simply the computer that aggregates the data and generates the corresponding graphs. The required software is in the *munin* package. The standard configuration runs `munin-cron` (once every 5 minutes), which gathers data from all the hosts listed in `/etc/munin/munin.conf` (only the local host is listed by default), saves the historical data in RRD files (*Round Robin Database*, a file format designed to store data varying in time) stored under `/var/lib/munin/` and generates an HTML page with the graphs in `/var/cache/munin/www/`.

All monitored machines must therefore be listed in the `/etc/munin/munin.conf` configuration file. Each machine is listed as a full section with a name matching the machine and at least an address entry giving the corresponding IP address.

```
[ftp.falcot.com]
    address 192.168.0.12
    use_node_name yes
```

Sections can be more complex, and describe extra graphs that could be created by combining data coming from several machines. The samples provided in the configuration file are good starting points for customization.

The last step is to publish the generated pages; this involves configuring a web server so that the contents of `/var/cache/munin/www/` are made available on a website. Access to this website will often be restricted, using either an authentication mechanism or IP-based access control. See section 11.2, "Web Server (HTTP)" page 293 for the relevant details.

12.4.2. Setting Up Nagios

Unlike Munin, Nagios does not necessarily require installing anything on the monitored hosts; most of the time, Nagios is used to check the availability of network services. For instance, Nagios can connect to a web server and check that a given web page can be obtained within a given time.

Installing

The first step in setting up Nagios is to install the *nagios4* and *monitoring-plugins* packages. Installing the packages configures the web interface and the Apache server. The authz_groupfile and auth_digest Apache modules must be enabled, for that execute:

```
# a2enmod authz_groupfile
Considering dependency authz_core for authz_groupfile:
Module authz_core already enabled
Enabling module authz_groupfile.
To activate the new configuration, you need to run:
  systemctl restart apache2
# a2enmod auth_digest
Considering dependency authn_core for auth_digest:
Module authn_core already enabled
Enabling module auth_digest.
To activate the new configuration, you need to run:
  systemctl restart apache2
# systemctl restart apache2
```

Adding other users is a simple matter of inserting them in the `/etc/nagios4/hdigest.users` file.

Pointing a browser at http://*server*/nagios4/ displays the web interface; in particular, note that Nagios already monitors some parameters of the machine where it runs. However, some interactive features such as adding comments to a host do not work. These features are disabled in the default configuration for Nagios, which is very restrictive for security reasons.

Enabling some features involves editing `/etc/nagios4/nagios.cfg`. We also need to set up write permissions for the directory used by Nagios, with commands such as the following:

```
# systemctl stop nagios4
# dpkg-statoverride --update --add nagios www-data 2710 /var/lib/nagios4/rw
# dpkg-statoverride --update --add nagios nagios 751 /var/lib/nagios4
# systemctl start nagios4
```

Configuring

The Nagios web interface is rather nice, but it does not allow configuration, nor can it be used to add monitored hosts and services. The whole configuration is managed via files referenced in the central configuration file, `/etc/nagios4/nagios.cfg`.

These files should not be dived into without some understanding of the Nagios concepts. The configuration lists objects of the following types:

- a *host* is a machine to be monitored;
- a *hostgroup* is a set of hosts that should be grouped together for display, or to factor some common configuration elements;

- a *service* is a testable element related to a host or a host group. It will most often be a check for a network service, but it can also involve checking that some parameters are within an acceptable range (for instance, free disk space or processor load);
- a *servicegroup* is a set of services that should be grouped together for display;
- a *contact* is a person who can receive alerts;
- a *contactgroup* is a set of such contacts;
- a *timeperiod* is a range of time during which some services have to be checked;
- a *command* is the command line invoked to check a given service.

According to its type, each object has a number of properties that can be customized. A full list would be too long to include, but the most important properties are the relations between the objects.

A *service* uses a *command* to check the state of a feature on a *host* (or a *hostgroup*) within a *timeperiod*. In case of a problem, Nagios sends an alert to all members of the *contactgroup* linked to the service. Each member is sent the alert according to the channel described in the matching *contact* object.

An inheritance system allows easy sharing of a set of properties across many objects without duplicating information. Moreover, the initial configuration includes a number of standard objects; in many cases, defining new hosts, services and contacts is a simple matter of deriving from the provided generic objects. The files in `/etc/nagios4/conf.d/` are a good source of information on how they work.

The Falcot Corp administrators use the following configuration:

Example 12.3 *`/etc/nagios4/conf.d/falcot.cfg` file*

```
define contact{
    name                            generic-contact
    service_notification_period     24x7
    host_notification_period        24x7
    service_notification_options    w,u,c,r
    host_notification_options       d,u,r
    service_notification_commands   notify-service-by-email
    host_notification_commands      notify-host-by-email
    register                        0 ; Template only
}
define contact{
    use             generic-contact
    contact_name    rhertzog
    alias           Raphael Hertzog
    email           hertzog@debian.org
}
define contact{
    use             generic-contact
```

```
    contact_name    rmas
    alias           Roland Mas
    email           lolando@debian.org
}

define contactgroup{
    contactgroup_name     falcot-admins
    alias                 Falcot Administrators
    members               rhertzog,rmas
}

define host{
    use                   generic-host ; Name of host template to use
    host_name             www-host
    alias                 www.falcot.com
    address               192.168.0.5
    contact_groups        falcot-admins
    hostgroups            debian-servers,ssh-servers
}
define host{
    use                   generic-host ; Name of host template to use
    host_name             ftp-host
    alias                 ftp.falcot.com
    address               192.168.0.6
    contact_groups        falcot-admins
    hostgroups            debian-servers,ssh-servers
}

# 'check_ftp' command with custom parameters
define command{
    command_name          check_ftp2
    command_line          /usr/lib/nagios/plugins/check_ftp -H $HOSTADDRESS$ -w 20 -c
        ➥  30 -t 35
}

# Generic Falcot service
define service{
    name                  falcot-service
    use                   generic-service
    contact_groups        falcot-admins
    register              0
}

# Services to check on www-host
define service{
    use                   falcot-service
    host_name             www-host
    service_description   HTTP
    check_command         check_http
```

```
}
define service{
    use                     falcot-service
    host_name               www-host
    service_description     HTTPS
    check_command           check_https
}
define service{
    use                     falcot-service
    host_name               www-host
    service_description     SMTP
    check_command           check_smtp
}

# Services to check on ftp-host
define service{
    use                     falcot-service
    host_name               ftp-host
    service_description     FTP
    check_command           check_ftp2
}
```

This configuration file describes two monitored hosts. The first one is the web server, and the checks are made on the HTTP (80) and secure-HTTP (443) ports. Nagios also checks that an SMTP server runs on port 25. The second host is the FTP server, and the check includes making sure that a reply comes within 20 seconds. Beyond this delay, a *warning* is emitted; beyond 30 seconds, the alert is deemed critical. The Nagios web interface also shows that the SSH service is monitored: this comes from the hosts belonging to the ssh-servers hostgroup. The matching standard service is defined in `/etc/nagios4/conf.d/services_nagios2.cfg`.

Note the use of inheritance: an object is made to inherit from another object with the "use *parent-name*". The parent object must be identifiable, which requires giving it a "name *identifier*" property. If the parent object is not meant to be a real object, but only to serve as a parent, giving it a "register 0" property tells Nagios not to consider it, and therefore to ignore the lack of some parameters that would otherwise be required.

DOCUMENTATION

List of object properties

A more in-depth understanding of the various ways in which Nagios can be configured can be obtained from the documentation hosted on https://assets.nagios.com/downloads/nagioscore/docs/nagioscore/4/en/index.html. It includes a list of all object types, with all the properties they can have. It also explains how to create new plugins.

GOING FURTHER

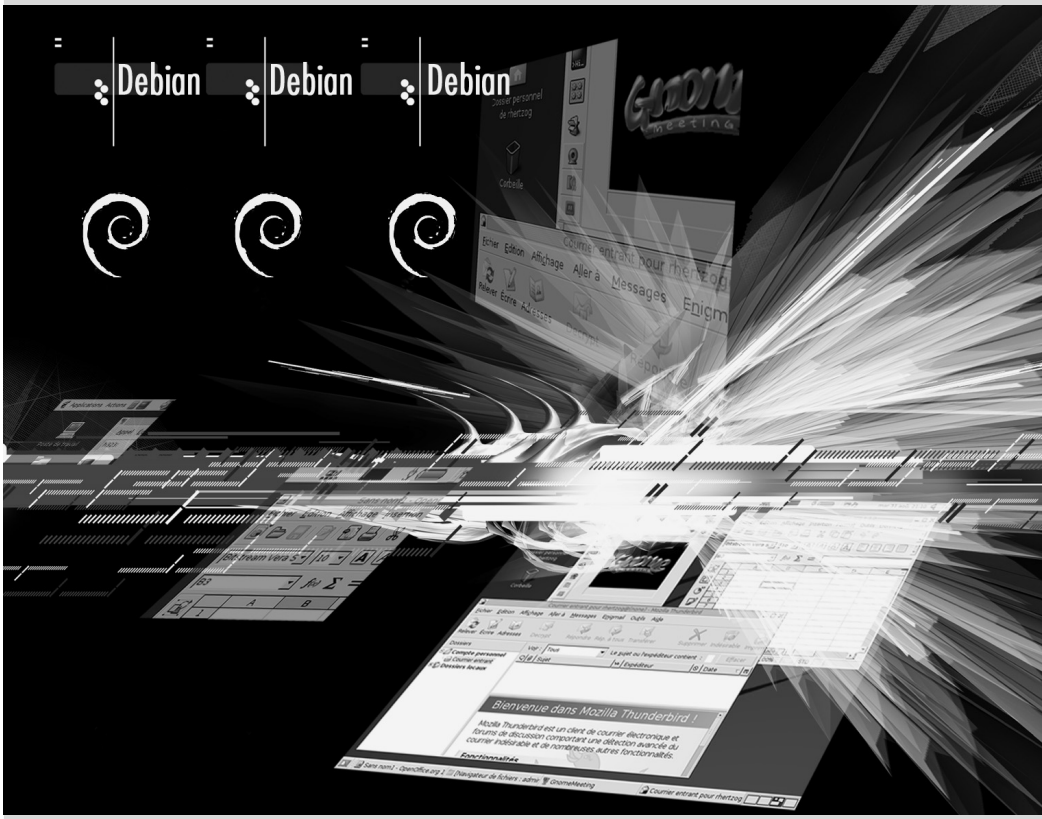
Remote tests with NRPE

Many Nagios plugins allow checking some parameters local to a host; if many machines need these checks while a central installation gathers them, the NRPE (*Nagios Remote Plugin Executor*) plugin needs to be deployed. The *nagios-nrpe-plugin* package needs to be installed on the Nagios server, and *nagios-nrpe-server* on the hosts where local tests need to run. The latter gets its configuration from

`/etc/nagios/nrpe.cfg`. This file should list the tests that can be started remotely, and the IP addresses of the machines allowed to trigger them. On the Nagios side, enabling these remote tests is a simple matter of adding matching services using the new *check_nrpe* command.

Keywords

Workstation
Graphical desktop
Office work
X.org



Chapter

13 Workstation

Contents

Configuring the X11 Server 382 Customizing the Graphical Interface 383 Graphical Desktops 385
Email 389 Web Browsers 391 Development 393 Collaborative Work 394 Office Suites 395
Emulating Windows: Wine 396 Real-Time Communications software 397

Now that server deployments are done, the administrators can focus on installing the individual workstations and creating a typical configuration.

13.1. Configuring the X11 Server

A brief reminder: X.org is the software component that allows graphical applications to display windows on screen. It includes a driver that makes efficient use of the video card. The features offered to the graphical applications are exported through a standard interface, *X11* (*Buster* contains version *X11R7.7*).

PERSPECTIVE

X11, XFree86 and X.org

X11 is the graphical system most widely used on Unix-like systems (also available for Windows and Mac OS). Strictly speaking, the term "X11" only refers to a protocol specification, but it is also used to refer to the implementation in practice.

X11 had a rough start, but the 1990s saw XFree86 emerge as the reference implementation because it was free software, portable, and maintained by a collaborative community. However, the rate of evolution slowed down near the end when the software only gained new drivers. That situation, along with a very controversial license change, led to the X.org fork in 2004. This is now the reference implementation, and Debian *Buster* uses X.org version 7.7.

Current versions of X.org are able to autodetect the available hardware: this applies to the video card and the monitor, as well as keyboards and mice; in fact, it is so convenient that the package no longer even creates a `/etc/X11/xorg.conf` configuration file.

The keyboard configuration is currently set up in `/etc/default/keyboard`. This file is used both to configure the text console and the graphical interface, and it is handled by the *keyboard-configuration* package. Details on configuring the keyboard layout are available in section 8.1.2, "Configuring the Keyboard" page 161.

The *xserver-xorg-core* package provides a generic X server, as used by the 7.x versions of X.org. This server is modular and uses a set of independent drivers to handle the many different kinds of video cards. Installing *xserver-xorg* ensures that both the server and at least one video driver are installed.

Note that if the detected video card is not handled by any of the available drivers, X.org tries using the vesa and fbdev drivers. VESA is a generic driver that should work everywhere, but with limited capabilities (fewer available resolutions, no hardware acceleration for games and visual effects for the desktop, and so on) while fbdev works on top of the kernel's framebuffer device. Nowadays the X server can run without any administrative privileges (this used to be required to be able to configure the screen) and its log file is then stored in the user's home directory in `~/.local/share/xorg/Xorg.0.log`, whereas it is `/var/log/Xorg.0.log` for X servers started with root privileges and for versions older than Debian 9 *Stetch*. That log file is where one would look to know what driver is currently in use. For example, the following snippet matches what the intel driver outputs when it is loaded:

```
(==) Matched intel as autoconfigured driver 0
(==) Matched modesetting as autoconfigured driver 1
(==) Matched vesa as autoconfigured driver 2
(==) Matched fbdev as autoconfigured driver 3
(==) Assigned the driver to the xf86ConfigLayout
```